

Objet, classe

- **Classe** : c'est le moule qui permet de fabriquer les objets (ex : la recette de pizza)
- **Objet** : c'est une réalisation concrète et utilisable de la classe. En jargon objet, on dit que c'est une « **instance** » de la classe (ex : la pizza)



Pointeur et référence

- **Pointeur** : correspond à une adresse mémoire qui contient une variable utilisé en C et C++ (ex : adresse d'une pizzeria)
- **Référence** : correspond à un alias pour une autre variable utilisé en java (ex : le nom de la pizzeria « Chez Tony »)



Les champs d'une classe

Aussi appelées **attributs** ou **données** ou **membres** ou **variables** de classe, sont des données typées (primitives ou structure de données) qui caractérisent une classe. Exemple avec un Pokemon : **String** nom, **int** pc, **int** pv, **enum** type...

Les méthodes d'une classe

Aussi appelées **opérations** ou **fonctions**, sont des fonctions appelables par un client et elles définissent le comportement ou une action de l'objet. Ces méthode peuvent recevoir des valeurs typées en paramètres qui sont appelés **arguments**, et elles peuvent retourner des valeurs typées. La description complète d'une méthode est appelée **signature**. Exemple avec un Pokemon : recharger (int pc), attaquer(). Pour une méthode qui ne retourne rien, on utilise le type **void**.

Il existe 3 types de méthodes :

- **Les constructeurs** : du nom que la classe et ne renvoie rien, cette dernière est exécutée qu'une seule fois pour instancier la classe (**new**)
- **Les accesseurs** : permettent l'accès et la modification des champs de classe (souvent privés). Getter (lecture), Setter (écriture).
- **Les fonctions** : les autres méthodes restantes.

L'encapsulation

C'est le mécanisme qui permet de définir des niveaux de visibilité des éléments de la classe. Cet accès aux données se définit par le biais de « modifieurs » parmi les choix suivants :

- **public** : donnée accessible à toutes les classes.
- **protected** : donnée réservée aux classes filles
- **private** : donnée limitée à la classe elle-même

Élément statique

Un élément statique (champs ou méthode), est un élément partagé entre toutes les instances d'une classe. Pour le préciser le mot clé **static** est spécifié. Exemple : la fonction main d'une classe est toujours **static**, il n'est alors pas nécessaire d'avoir une instance de classe pour y accéder.

```
public class ExempleJava {

    protected static String maDonneeStatique = "donnees statique";

    private static void maMethodeStatique (int argument){
        System.out.println("maMethodeStatique=" + argument);
    }

    public static void main(String[] args) {
        //Affectation d une valeur a un champs statique, il n est pas necessaire d avoir une instance
        ExempleJava.maDonneeStatique = "Nouvelle valeur";

        //Appel d une methode statique, il n est pas necessaire d avoir une instance
        ExempleJava.maMethodeStatique(8);
    }
}
```

Héritage

L'héritage permet d'enrichir des classes existantes qui hériteront du savoir-faire de leur parent.

Dans cet exemple les classes **filles** ou **enfants** Têtarte, Miaouss, Salamèche héritent de la classe **mère** ou **parent** Pokémon. La classe parent est aussi appelée la **superclasse**.

Pour définir qu'une classe hérite d'une autre on utilise le mot clé **extends**



Les références this et super.

- **this** : sert à désigner l'instance courante de la classe dans laquelle elle est utilisée. Ex : this.maFonction() appelle maFonction de l'instance courante.
- **super** : sert à désigner la superclasse. Ex : super.toString() appelle la méthode toString de la classe parente.

Interface

En JAVA, l'héritage multiple n'existe pas mais les **interfaces** en reprennent les principaux avantages. Les interfaces traduisent la capacité d'un objet à adopter un comportement en plus de ce que son parent peut lui fournir. On spécifie une interface avec le mot clé **interface**. Afin qu'une classe adopte le comportement d'une interface, on implémente l'interface avec le mot clé **implements**. Toutes les méthodes d'une interface sont abstraites, cela signifie que la méthode ne possède pas de corps.

Définition d'une interface

```
package fr.ifadelorozoy;

public interface IPokemon {

    public int attaquer();

    public void recharger (int pc);
}
```

Implémentation d'une interface

```
package fr.ifadelorozoy;

public class Pikachu implements IPokemon {

    @Override
    public int attaquer() {
        return 0;
    }

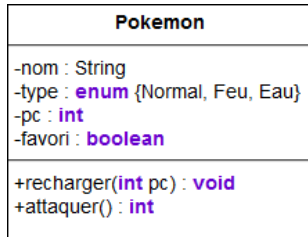
    @Override
    public void recharger(int pc) {
    }
}
```

Introduction

Expriment de manière générale la structure statique d'un système.
 La classe regroupe un ensemble d'objets ayant les mêmes propriétés et les mêmes comportements.
 Les propriétés déterminent les états possibles de l'objet.
 Les généralités sont contenues dans la classe et les particularités dans les objets.

Une classe

Représentation graphique



+ : Élément public (public)
 # : Élément protégé (protected)
 - : Élément privé (private)

Implémentation

```
package fr.ifadelorozoy;

public class Pokemon {

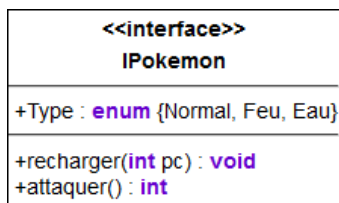
    public static enum Type {Normal, Feu, Eau};
    private String nom;
    private Type type = Type.Normal;
    private int pc;

    public void recharger(int pc){
        this.pc = this.pc + pc;
    }

    public int attaquer(){
        return 0;
    }
}
```

Une interface

Représentation graphique



Implémentation

```
package fr.ifadelorozoy;

public interface IPokemon {

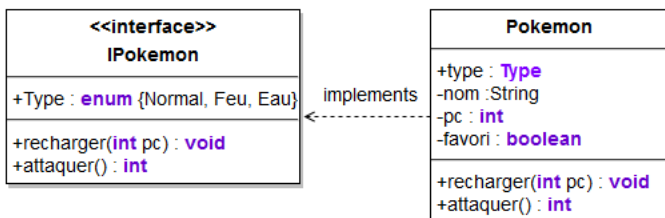
    public static enum Type {Normal, Feu, Eau};

    public void recharger (int pc);

    public int attaquer();
}
```

L'implémentation d'une interface

Représentation graphique



Implémentation

```
package fr.ifadelorozoy;

public class Pokemon implements IPokemon {

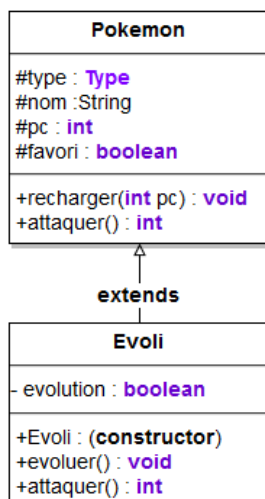
    private String nom;
    private Type type = Type.Normal;
    private int pc;
    private boolean favori = false;

    public void recharger(int pc){
        this.pc = this.pc + pc;
    }

    public int attaquer(){
        return 0;
    }
}
```

L'héritage

Représentation graphique



Implémentation

```
package fr.ifadelorozoy;

public class Evoli extends Pokemon {

    private boolean evolution = true ; //Ce pokemon peut evoluer

    //Surcharge du constructeur
    public Evoli() {
        super(); //Appel le constructeur parent
        nom = "Evoli"; //Initialise le nom
    }

    //Surcharge de la methode attaquer
    @Override
    public int attaquer() {
        System.out.println("Attaque evoli");
        return super.attaquer(); //Appel de la methode parent
    }

    public void evoluer(){
        //Code pour faire evoluer Evoli
        nom = "Aquali";
        this.pc = this.pc + 100;
        evolution = false ; //Ce pokemon ne peut plus evoluer
    }
}
```